

CRAFT: Corrective and Robust Multi-Agent Framework for Text-to-Parametric CAD

Mohammed Musthafa Rafi^a Anushrut Jignasu^a Mahdi Saraeian^a Chinmay Hegde^b Aditya Balu^a
Adarsh Krishnamurthy^{a*}

^aIowa State University, USA ^bNew York University, USA

Email: ^a (mohd7 — anushrutg — mehdi — baditya — adarsh)@iastate.edu , ^b chinmay.h@nyu.edu

Abstract—Text-to-CAD generation has the potential to make mechanical CAD more accessible, but existing approaches face a trade-off between data dependence and output fidelity. Supervised methods rely on large paired text–model datasets, while zero-shot prompting strategies are often brittle and tend to lose the parametric structure needed for downstream editing and customization. We present CRAFT, a corrective and robust multi-agent framework that produces parametric OpenSCAD programs from natural language without task-specific training. At its core is a JSON intermediate representation that carries symbolic parametric expressions throughout the generation process, so the resulting models remain editable and retain their parametric structure. CRAFT combines semantic understanding, parametric planning, compilation, rendering, self-correction, and component-level verification within a layered recovery process, using multi-view visual feedback to detect and repair geometric errors. Across NopSCADlib, ABC, and Slice-100K, CRAFT is competitive with direct zero-shot baselines in perceptual and geometric quality while exposing substantially more editable parametric structure.

Index Terms—Text-to-CAD, parametric modeling, multi-agent systems, large language models, zero-shot generation, OpenSCAD

I. INTRODUCTION

Text-driven geometric modeling has seen rapid progress in recent years, with large language and vision models enabling increasingly capable systems for generating 3D shapes directly from natural language descriptions [1–4]. These advances have raised the prospect of making mechanical computer-aided design (CAD) accessible to users without specialized CAD expertise, reducing the barrier between a design intent expressed in words and a manufacturable geometric model. Despite this progress, existing approaches remain brittle in practice. Supervised methods that fine-tune models on paired text–CAD datasets can produce geometrically plausible outputs within their training distribution. They generalize poorly to novel shapes, however, and require substantial curated data to train [5–8]. Zero-shot prompting strategies avoid this data dependency but are prone to generating syntactically invalid code, non-renderable geometry, and outputs that fail to faithfully reflect the original prompt. Both families of approaches also tend to produce geometry with fixed numeric dimensions rather than editable parametric models, limiting their utility for downstream engineering and customization tasks [9, 10].

Agentic systems offer a natural path toward addressing these limitations [11, 12]. Rather than treating CAD generation as a single forward pass, a multi-agent pipeline can decompose the problem into discrete stages, including semantic understanding, structured planning, code synthesis, and verification, each of which can be independently validated and repaired. This staged architecture allows errors to be detected and corrected locally rather than propagating silently into the final output, improving both reliability and geometric fidelity. A particularly well-suited target for such a pipeline is OpenSCAD [13], a declarative scripting language in which geometry is expressed as a program over symbolic parameters. Generating OpenSCAD code rather than mesh geometry preserves the parametric relationships that make CAD models editable and reusable. However, verifying the correctness of generated OpenSCAD programs is difficult: a program may compile successfully yet produce geometry that is visually or dimensionally incorrect relative to the prompt. Effective verification therefore requires grounding outputs in external reference knowledge, both to check that recognized components conform to standard specifications and to confirm that expected parts are present. NopSCADlib [14], a parametric OpenSCAD component library, provides catalog-defined specifications for standard mechanical parts, grounding outputs in physically meaningful dimensions and enabling verification of component completeness.

We present CRAFT (Fig. 1), a corrective and robust multi-agent framework for text-to-parametric CAD generation. CRAFT produces executable and editable OpenSCAD programs from natural language without task-specific training through three core mechanisms: a JSON-based intermediate representation that preserves symbolic parametric expressions, multi-view visual feedback to detect and correct geometric errors, and a layered recovery strategy to handle failures at every stage of the pipeline. Our main contributions are as follows:

- We develop a corrective and robust six-stage multi-agent framework for zero-shot text-to-parametric CAD generation that combines structured planning, multi-view visual verification, and layered error recovery to produce executable OpenSCAD programs from natural language.
- We propose a JSON-based intermediate representation that preserves symbolic parametric expressions throughout the generation process, enabling the produced CAD

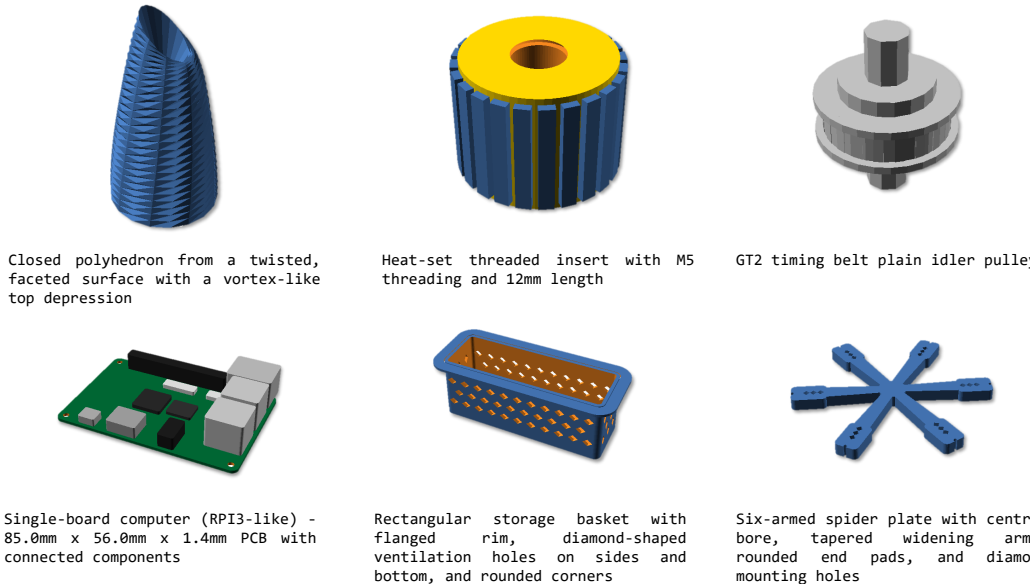


Fig. 1: Examples of different CAD models created by CRAFT from the prompts.

programs to remain editable and reusable rather than hardening into fixed numeric geometry.

- We evaluate CRAFT across three datasets (NopSCADlib, ABC, and Slice-100K) and show that it achieves competitive perceptual and geometric quality and substantially greater editability relative to direct generation baselines, and we isolate the contribution of each framework component through an ablation study.

Our implementation, generated OpenSCAD outputs, and evaluation scripts are publicly available at <https://github.com/idealab-isu/CRAFT>.

II. RELATED WORK

Text-to-3D and Text-to-CAD Generation. Generative approaches to 3D modeling have advanced rapidly, spanning neural implicit representations [2], point cloud synthesis [4], and shape generation from text [3]. While these methods produce visually compelling outputs, they typically yield fixed mesh geometry that cannot be edited parametrically or reused in engineering workflows. A parallel line of work targets CAD-specific representations: Text2CAD’s sequential construction sequences [5], LLM4CAD’s multimodal generation [8], CAD-Coder’s chain-of-thought fine-tuning with geometric rewards [6], and Text-to-CadQuery’s scaled paired-data training [7]. These supervised approaches achieve strong in-distribution results but require curated paired training data and produce outputs that are often not directly editable as parametric programs. CRAFT avoids task-specific training entirely and instead uses structured zero-shot generation to produce symbolic, editable OpenSCAD programs.

Program Synthesis and Code Generation. Generating executable code from natural language has become a well-studied problem, with large language models demonstrating strong performance across programming tasks [15]. Tech-

niques such as chain-of-thought prompting [16] and self-refinement [12] have shown that decomposing generation into reasoning steps improves correctness on structured outputs, motivating intermediate representations and iterative repair for CAD, where generated programs must satisfy syntactic validity, successful compilation, and geometric and semantic correctness. CRAFT operationalizes this through a JSON intermediate representation that decouples planning from code synthesis, enabling schema-level validation before any OpenSCAD code is produced.

Verification and Visual Feedback. Recent work addresses correctness verification as a first-class concern in CAD generation. CADCodeVerify [17] introduces vision-language model-based verification and iterative correction as a post-generation step, improving quality without retraining. More broadly, visual feedback has emerged as a powerful signal for program repair, providing information difficult to obtain from code analysis alone. CRAFT extends this by rendering six orthographic views, enabling the VLM to detect errors not visible from any single view, and integrating this within a bounded correction loop instead of a post-processing step.

Retrieval-Augmented Generation. Retrieval-augmented generation (RAG) grounds language model outputs in external knowledge, improving factual accuracy and domain specificity [18], and in code generation reduces hallucination by retrieving relevant library components at generation time. CRAFT applies this to parametric CAD by retrieving component specifications from NopSCADlib [14] during the understanding stage, grounding geometry in catalog-defined dimensions rather than the model’s implicit knowledge.

Multi-Agent Systems and Parametric Modeling. Multi-agent pipelines outperform single-model generation on tasks requiring planning, execution, and verification across multiple steps [11], isolating failure modes and applying targeted repair

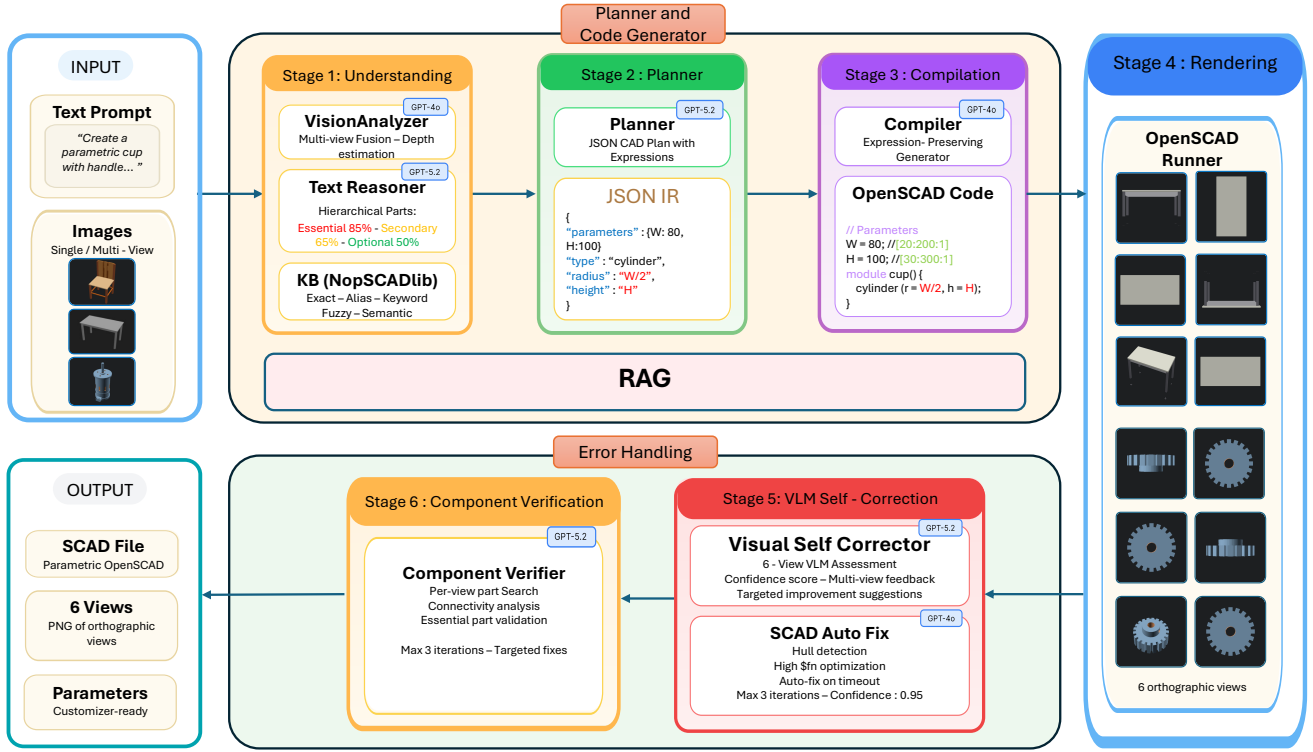


Fig. 2: CRAFT maps a text prompt and optional reference images to a parametric OpenSCAD program through six stages: understanding, planning, compilation, rendering, vision-language model (VLM) self-correction, and component verification. Reasoning-intensive stages use GPT-5.2, while deterministic translation and repair steps use GPT-4o.

without regenerating earlier stages. This suits CAD generation, where failures arise at the planning, compilation, rendering, or geometric-correctness levels, each requiring a different repair strategy. The target representation matters as much as the architecture: parametric CAD expresses geometry through symbolic variables and editable relationships rather than fixed coordinates, enabling downstream customization, reuse, and analysis [9, 10]. Most text-to-CAD systems emit hard-coded numeric geometry; CRAFT instead stores dimensions as symbolic strings in its JSON intermediate representation and carries them through compilation unchanged, so the final OpenSCAD [13] programs remain fully editable.

III. METHODS

CRAFT transforms natural language descriptions and optional reference images into parametric OpenSCAD code through six sequential stages. In our implementation, reasoning-intensive stages (text understanding, planning, multi-view assessment, and component verification) are handled by a reasoning large language model (LLM, GPT-5.2) [19], while image analysis and deterministic translation and repair steps are handled by a traditional LLM (GPT-4o) [20]. The six stages describe the forward generation path, whereas the five recovery layers introduced in Section III-C describe how the system escalates repair when intermediate artifacts are invalid or the rendered result is unsatisfactory.

The framework (Fig. 2) begins with understanding (Stage 1), where image and text inputs are processed in parallel. The image-analysis path extracts geometric cues from any provided reference images and fuses information across views when multiple images are available. The text-reasoning parses the prompt to identify required components, classify them by importance (essential, secondary, optional), and detect references to standard parts such as motors or fasteners. When such components are recognized, their specifications are retrieved from NopSCADlib [14] using five strategies (exact name matching, curated alias lookup, keyword-based lookup, fuzzy string matching, and semantic embedding retrieval), following a retrieve-then-generate design pattern [18]. If retrieval fails, the framework proceeds with LLM-only synthesis.

The output of stage 1 is passed to planning (Stage 2), where the planner produces a JSON CAD plan (Section III-A). This intermediate representation encodes geometry, constructive solid geometry (CSG) operations, and symbolic parametric expressions in a structured format that can be validated before any code is generated. In compilation (Stage 3), the validated plan is translated into OpenSCAD that preserves expression strings and generates parameter ranges for the customizer interface.

In rendering (Stage 4), OpenSCAD produces six orthographic views (front, back, left, right, top, bottom), applying timeout-aware simplifications for expensive constructs such as `hull()`. These views drive VLM self-correction

(Stage 5, [Section III-B](#)). Finally, component verification (Stage 6) checks whether expected parts are present using tiered confidence thresholds of 85% for essential parts, 65% for secondary, and 50% for optional. These thresholds were set empirically during pilot runs to prioritize functionally necessary components over cosmetic details, a process related to recent self-refinement approaches for large models [12].

A. JSON Intermediate Representation

CRAFT uses a JSON intermediate representation (IR) between design understanding and OpenSCAD generation. The IR encodes parameters, geometric primitives, and CSG operations in a structured form that can be validated before compilation. The IR preserves symbolic parametric expressions and defers their evaluation: expressions such as "outer_diameter/2" pass unchanged into the compiled OpenSCAD code, allowing the final model to remain editable and fully parametric.

B. VLM Self-Correction

Single-image assessment can miss geometric issues visible only from specific viewpoints: a model that looks correct from the front may have wrong depth or features visible only from the side. CRAFT addresses this by rendering six orthographic views and presenting them jointly to the VLM for assessment. The VLM returns a confidence score on a 0–1 scale, a list of identified issues, and suggested corrections. If the confidence falls below 0.95, the system generates targeted fixes for the affected parts rather than regenerating the entire design. This loop runs for up to three iterations, with early stopping once the confidence threshold is reached. In our implementation, this assessment is performed using GPT-5.2 due to its stronger spatial reasoning capability.

C. Layered Error Recovery

The recovery layers describe how CRAFT escalates repair when intermediate artifacts fail validation, rendering, or semantic checks. Failures in text-to-CAD generation can arise at several levels: invalid intermediate plans, non-renderable code, and geometrically incorrect outputs. CRAFT addresses these through a five-layer strategy that orders repair mechanisms from inexpensive structural fixes to more costly semantic correction. The first layer repairs malformed JSON plans through schema validation and automatic repair. The second layer applies deterministic OpenSCAD fixes for common rendering failures, such as expensive operations or excessive mesh resolution. The third layer re-invokes the multi-view VLM assessment from Stage 5 to detect semantic and geometric mismatches and generate targeted corrections. The fourth layer re-invokes the component verification procedure from Stage 6 under the same tiered thresholds. The fifth layer incorporates manual repair hints when automatic recovery remains insufficient. Automatic recovery operates under a fixed retry budget of 10 total attempts across the first four layers, set empirically to balance correction capability against latency and cost. In the reported experiments, staged recovery resolved compilation

and rendering failures before final output assessment, allowing the comparison to focus on geometric fidelity and semantic alignment of the final recovered outputs.

IV. EXPERIMENTAL SETUP

Datasets. We evaluate CRAFT on three datasets totaling 668 components to assess both in-domain performance and cross-dataset generalization. The primary benchmark is NopSCADlib [14], comprising 468 text–CAD pairs derived from a parametric OpenSCAD mechanical component library, with ground-truth geometry from catalog-defined parametric specifications. To test generalization beyond the retrieval library, we also evaluate on 100 shapes each from ABC [21] and Slice-100K [22]; these lie outside NopSCADlib and are run with retrieval disabled. Within each dataset, components are split into Simple, Medium, and Complex terciles by a weighted composite of structural features (NopSCADlib source code) or mesh-geometry features (external meshes); because the feature spaces differ, tiers are relative within a dataset rather than comparable across datasets.

Baselines and configuration. We compare against direct generation with GPT-4o [20] and GPT-5.2 [19], which receive the same prompt and a system instruction to emit only valid OpenSCAD code in a single deterministic (temperature 0.0) call, with no intermediate planning, retrieval, visual feedback, or recovery. This evaluates CRAFT against standard one-shot direct generation from the same model family, although the methods use different inference budgets. CRAFT uses a hybrid allocation (GPT-5.2 for reasoning-intensive stages and GPT-4o for deterministic compilation and repair), with up to 2 schema-repair attempts, up to 3 VLM self-correction iterations (early stopping at confidence > 0.95), and up to 3 verification iterations under the tiered thresholds of [Section III](#).

Metrics and protocol. For geometric evaluation we report bidirectional Chamfer Distance (CD), F1 at a 1% distance threshold, and voxel Intersection-over-Union (IoU) on a 64^3 grid, computed from point clouds of 10,000 points sampled per mesh:

$$CD = \frac{1}{2} (\text{mean}(d_{\text{gt} \rightarrow \text{pred}}) + \text{mean}(d_{\text{pred} \rightarrow \text{gt}})). \quad (1)$$

For perceptual evaluation we report CLIP score (ViT-L/14 [23]) for text–image alignment and Fréchet Inception Distance (FID [24]) from Inception-v3 features for distributional similarity to ground-truth renders. All views are rendered at 512×512 resolution. When a method fails to produce a valid output, the example is assigned worst-case metric values so that invalid generations remain reflected in the aggregates.

V. RESULTS

We evaluate CRAFT against direct generation baselines (GPT-4o and GPT-5.2) across semantic alignment, geometric faithfulness, and editability, and ablate each component to isolate its contribution, spanning the in-domain NopSCADlib benchmark and the two external datasets of [Section IV](#).

TABLE I: *Perceptual evaluation on NopSCADlib. CLIP Score measures text–image alignment (higher is better), while FID measures distributional similarity to ground-truth renders (lower is better). GT denotes the ground-truth CLIP score as a reference value.*

Metric	CRAFT	GPT-4o	GPT-5.2	GT
CLIP Score	0.2226	0.2033	0.2145	0.2342
Simple	0.2272	0.2098	0.2174	0.2279
Medium	0.2132	0.2014	0.2144	0.2342
Complex	0.2279	0.1983	0.2116	0.2409
FID	95.11	119.42	106.22	—
Simple	152.37	144.38	136.22	—
Medium	126.88	159.56	147.95	—
Complex	144.85	184.39	160.51	—

A. Perceptual Alignment

Table I reports perceptual alignment. CRAFT attains the highest overall CLIP score, closest to the ground-truth reference, and the lowest overall FID. On FID, the margin is most pronounced on medium and complex components, while on simple parts GPT-5.2 attains the lowest FID, consistent with direct generation being effective for basic geometry. On CLIP score, CRAFT leads on simple and complex components, with GPT-5.2 narrowly ahead on medium. The Overall FID is lower than any per-tier value because it is pooled over all 468 renders rather than averaged across tiers; FID is non-additive, so the pooled estimate need not lie between tier values and carries less finite-sample bias.

B. Geometric Accuracy

We next evaluate geometric fidelity using Chamfer Distance, F1 (at a 1% threshold), and voxel IoU. Table II reports in-domain results on NopSCADlib and Table III the out-of-domain results on ABC and Slice-100K. On NopSCADlib, CRAFT attains the best overall Chamfer distance and the best overall voxel IoU and is comparable to the strongest baseline on F1. Direct generation remains competitive on the simpler in-distribution parts, whereas CRAFT’s advantage is clearest on volumetric overlap and on the more complex components. We therefore characterize CRAFT’s in-domain geometry as competitive and tied-or-better rather than dominant. On the external datasets, where retrieval is disabled, CRAFT generalizes well: it is best on every metric on ABC and best on Chamfer distance and F1 on Slice-100K, with GPT-4o edging voxel IoU on the latter. Taken together with the perceptual results, these findings indicate that CRAFT produces geometrically coherent parametric models with balanced quality across metrics. These numbers reflect end-to-end quality after the layered recovery of Section III-C, whose per-stage behavior we quantify in Section V-E.

C. Ablation Study

We disable each framework component in turn on the full NopSCADlib benchmark to understand the effect of the

TABLE II: *Geometric evaluation on NopSCADlib, including overall results and breakdown by prompt complexity. F1 is reported at a 1% distance threshold. Best results in each row group are shown in bold.*

Split	Method	CD↓	F1↑	Voxel IoU↑
Overall	CRAFT	0.0654	0.2530	0.2356
	GPT-4o	0.0733	0.2467	0.2135
	GPT-5.2	0.0662	0.2587	0.1890
Simple	CRAFT	0.0805	0.3771	0.2863
	GPT-4o	0.0841	0.3399	0.3339
	GPT-5.2	0.0800	0.3475	0.1554
Medium	CRAFT	0.0594	0.2277	0.2431
	GPT-4o	0.0652	0.2262	0.1810
	GPT-5.2	0.0571	0.2357	0.2591
Complex	CRAFT	0.0559	0.1492	0.1744
	GPT-4o	0.0706	0.1694	0.1201
	GPT-5.2	0.0614	0.1883	0.1487

TABLE III: *Geometric evaluation on the external datasets (ABC and Slice-100K), with retrieval disabled. F1 is reported at a 1% distance threshold. Best per dataset in bold.*

Dataset	Method	CD↓	F1↑	Voxel IoU↑
ABC	CRAFT	0.0841	0.1233	0.0433
	GPT-4o	0.1008	0.0857	0.0212
	GPT-5.2	0.0969	0.0875	0.0192
Slice-100K	CRAFT	0.0612	0.2924	0.0616
	GPT-4o	0.0751	0.2122	0.0711
	GPT-5.2	0.0622	0.2768	0.0636

removal of each component relative to the baseline (full framework). Removing retrieval (–retrieval) produces the largest degradation on voxel IoU, confirming the value of catalog-grounded component specifications for in-domain generation. Removing layered recovery (–recovery) produces the largest degradation on Chamfer distance and F1, and removing VLM self-correction (–multi-view) produces a comparable drop in F1 with a smaller increase in Chamfer distance. Removing the JSON intermediate representation (–JSON-IR) yields a similar Chamfer distance but lower F1 and voxel IoU; its primary role is to preserve the symbolic structure underlying CRAFT’s editability (Section V-D) rather than to improve raw geometry. Removing component verification (–verification) slightly improves the mesh-based metrics, indicating that these metrics do not directly capture the semantic-completeness objective of the verification stage, which adds or repairs prompt-specified parts.

D. Editability

A central goal of CRAFT is to produce editable parametric programs rather than fixed geometry. We quantify this by perturbing the exposed top-level parameters of each generated program by $\pm 10\%$ and $\pm 50\%$ and re-rendering. The *symbolic-preservation rate* is the fraction of secondary assignments whose value is an expression referencing other parameters

TABLE IV: Editability on NopSCADlib: exposed parameter count, symbolic-preservation rate, edit-success rate, and post-edit render validity.

Method	# Params	Symbolic $\uparrow$	Edit succ. $\uparrow$	Valid $\uparrow$
CRAFT	13.1	66.8%	99.9%	99.1%
GPT-5.2	4.2	47.7%	99.2%	97.7%
GPT-4o	0.1	3.0%	95.0%	95.0%

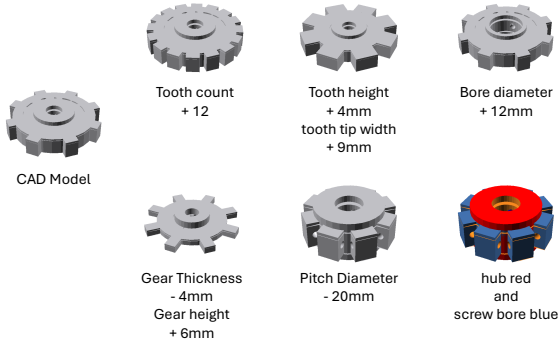


Fig. 3: Preserved parametric editability on a generated spur gear: single- and multi-parameter edits produce coherent geometric and appearance changes. The bottom-right panel labels the hub (red) and screw bores (blue) for reference.

rather than a hard-coded literal; *edit success* is the fraction of (parameter, perturbation) attempts that apply, and *post-edit validity* is the fraction whose perturbed program still renders to a valid, non-empty mesh. Table IV shows that CRAFT exposes 13.1 editable parameters per model on average, against 4.2 for GPT-5.2 and 0.1 for GPT-4o, and preserves symbolic expressions in 66.8% of cases versus 47.7% and 3.0%. Edit success and post-edit validity are uniformly high across methods, so the methods differ chiefly in how much editable structure they expose; edits apply cleanly regardless. The advantage widens with design difficulty: CRAFT’s symbolic-preservation rate rises from 59.2% on simple to 78.1% on complex parts, with mean exposed parameters increasing from 10.2 to 17.6. Fig. 3 shows representative single- and multi-parameter edits on a generated spur gear.

E. Recovery Behavior

Table V reports per-stage recovery on NopSCADlib. The two inexpensive structural layers (schema repair, SCAD auto-fix) are rarely triggered but always succeed when invoked. VLM self-correction is the most frequently used layer, addressing 80.4% of samples and resolving about half of those. Component verification is triggered on 17.3% of samples and applies part-level corrections; because its objective is component completeness rather than the rendered-image confidence used to score the other layers, its recovery rate under that shared success flag is 0 (Table V, note *). The manual-repair layer was never invoked, confirming that recovery was fully automatic across the benchmark.

TABLE V: Per-stage recovery on NopSCADlib. Trigger rate is the fraction of samples invoking a layer; recovery rate is its post-recovery success rate; retries is the mean number of attempts when invoked. The manual-repair layer is omitted: it was never invoked in these experiments.

Layer	Trigger	Recovery	Retries
schema repair	0.072	1.000	1.00
SCAD auto-fix	0.007	1.000	1.00
VLM self-correction	0.804	0.503	1.62
component verification	0.173	0.000*	1.61

*Verification is scored by part presence, not the shared success flag (essential/secondary pass: 94.6%/97.2%).

VI. DISCUSSION

CRAFT achieves competitive perceptual and geometric quality and much greater editability than direct one-shot generation, with the clearest gains on complex and out-of-distribution designs. This supports the view that structured generation becomes more valuable as geometric dependencies and design complexity grow. The JSON intermediate representation is designed to preserve symbolic dependencies during code generation; its ablation produces a similar Chamfer distance but lower F1 and voxel IoU, indicating that it also affects geometric structure. The editability evaluation further shows that the complete CRAFT framework exposes considerably more parameters and symbolic relationships than direct generation, although isolating the JSON IR’s contribution to editability would require an editability-specific ablation.

Several **limitations** should be noted. Performance declines as complexity increases, particularly when shapes involve many interacting parts or implicit structural relationships that are difficult to infer from text alone. OpenSCAD, while well suited for symbolic parametric modeling, does not capture the feature-history richness of industrial CAD systems such as Fusion 360 Gallery [10]. The multi-stage framework also depends on external language and vision models, increasing inference cost and introducing the risk of error propagation across stages. Finally, the evaluation covers three datasets, which represents only a subset of real-world designs.

VII. CONCLUSIONS

We presented CRAFT, a multi-agent framework for text-to-parametric CAD generation that combines structured reasoning, a JSON-based intermediate representation, visual verification, and layered error recovery. CRAFT explicitly preserves symbolic parametric relationships during generation, producing OpenSCAD programs that are executable, editable, and reusable. Experiments across three datasets show that CRAFT achieves competitive perceptual and geometric quality and markedly higher editability, and an ablation isolates retrieval as the dominant contributor to in-domain volumetric overlap, with layered recovery contributing most to Chamfer distance and F1. These results demonstrate that explicitly preserving parametric intent is a promising and practical direction for text-to-CAD systems.

REFERENCES

- [1] L. Zhang, B. Le, N. Akhtar, S.-K. Lam, and T. Ngo, "Large language models for computer-aided design: A survey," *ACM Computing Surveys*, 2026. [Online]. Available: <https://doi.org/10.1145/3787499>
- [2] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall, "DreamFusion: Text-to-3D using 2D diffusion," in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2209.14988>
- [3] H. Jun and A. Nichol, "Shap-E: Generating conditional 3D implicit functions," 2023. [Online]. Available: <https://arxiv.org/abs/2305.02463>
- [4] A. Nichol, H. Jun, P. Dhariwal, P. Mishkin, and M. Chen, "Point-E: A system for generating 3D point clouds from complex prompts," 2022. [Online]. Available: <https://arxiv.org/abs/2212.08751>
- [5] M. S. Khan, A. Sinha, T. U. Sheikh, D. Stricker, S. A. Ali, and M. Z. Afzal, "Text2CAD: Generating sequential CAD models from beginner-to-expert level text prompts," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/0e5b96f97c1813bb75f6c28532c2ecc7-Paper-Conference.pdf
- [6] Y. Guan, X. Wang, X. Xing, J. Zhang, D. Xu, and Q. Yu, "CAD-Coder: Text-to-CAD generation with chain-of-thought and geometric reward," in *Advances in Neural Information Processing Systems*, 2025.
- [7] H. Xie and F. Ju, "Text-to-CadQuery: A new paradigm for CAD generation with scalable large model capabilities," arXiv preprint, 2025. [Online]. Available: <https://arxiv.org/abs/2505.06507>
- [8] X. Li, Y. Sun, and Z. Sha, "LLM4CAD: Multimodal large language models for three-dimensional computer-aided design generation," *Journal of Computing and Information Science in Engineering*, vol. 25, no. 2, p. 021005, 2025.
- [9] R. Wu, C. Xiao, and C. Zheng, "DeepCAD: A deep generative network for computer-aided design models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 6772–6782. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2021/html/Wu_DeepCAD_A_Deep_Generative_Network_for_Computer-Aided_Design_Models_ICCV_2021_paper.html
- [10] K. D. D. Willis, Y. Pu, J. Luo, H. Chu, T. Du, J. G. Lambourne, A. Solar-Lezama, and W. Matusik, "Fusion 360 gallery: A dataset and environment for programmatic CAD construction from human design sequences," *ACM Transactions on Graphics*, vol. 40, no. 4, pp. 54:1–54:24, 2021.
- [11] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, 2024, pp. 8052–8060.
- [12] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://openreview.net/forum?id=S37hOerQLB>
- [13] M. Kintel and C. Wolf, "OpenSCAD: The programmers' solid 3D CAD modeller," 2024, website. [Online]. Available: <https://openscad.org>
- [14] nophead, "NopSCADlib: An ever expanding library of parts for OpenSCAD," 2024, gitHub repository. [Online]. Available: <https://github.com/nophead/NopSCADlib>
- [15] M. Chen, J. Tworek, H. Jun *et al.*, "Evaluating large language models trained on code," arXiv preprint, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [16] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [17] K. Alrashedy, P. Tambwekar, Z. Zaidi, M. Langwasser, W. Xu, and M. Gombolay, "Generating CAD code with vision-language models for 3D designs," 2024. [Online]. Available: <https://arxiv.org/abs/2410.05340>
- [18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [19] OpenAI, "Update to GPT-5 system card: GPT-5.2," Dec. 2025, openAI Deployment Safety Hub. [Online]. Available: <https://openai.com/index/gpt-5-system-card-update-gpt-5-2/>
- [20] —, "GPT-4o system card," 2024. [Online]. Available: <https://arxiv.org/abs/2410.21276>
- [21] S. Koch, A. Matveev, Z. Jiang, F. Williams, A. Artemov, E. Burnaev, M. Alexa, D. Zorin, and D. Panozzo, "ABC: A big CAD model dataset for geometric deep learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9601–9611. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/html/Koch_ABC_A_Big_CAD_Model_Dataset_for_Geometric_Deep_Learning_CVPR_2019_paper.html
- [22] A. Jignasu, K. O. Marshall, A. K. Mishra, L. N. Rillo, B. Ganapathysubramanian, A. Balu, C. Hegde, and A. Krishnamurthy, "Slice-100K: A multimodal dataset for extrusion-based 3D printing," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.04180>
- [23] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning*, ser. PMLR, vol. 139, 2021, pp. 8748–8763.
- [24] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.